

ANHANGUERA-LONDRINA/PR

ENGENHARIA DE SOFTWARE

PORTFÓLIO - RELATÓRIO DE AULA PRÁTICA:

NOME DA DISCIPLINA: FRAMEWORKS PARA DESENVOLVIMENTO DE SOFTWARE

Londrina/PR
2026

PORTFÓLIO - RELATÓRIO DE AULA PRÁTICA

NOME DA DISCIPLINA: FRAMEWORKS PARA DESENVOLVIMENTO DE SOFTWARE

Trabalho de portfólio apresentado como requisito parcial para a obtenção de pontos para a média semestral.

Orientador: Tutor

SUMÁRIO

1. INTRODUÇÃO	**4**
2. DESENVOLVIMENTO	**5**
2.1 AMBIENTE E ADAPTAÇÃO PARA MACOS	**5**
2.2 ATIVIDADE 1 - SPRING MVC E TOMCAT	**6**
2.3 ATIVIDADE 2 - CONSULTA DE CEP	**8**
2.4 ATIVIDADE 3 - FORMULÁRIO SPRING MVC	**10**
2.5 ATIVIDADE 4 - JQUERY E VIACEP	**12**
2.6 ANÁLISE DOS RESULTADOS	**14**
3. CONCLUSÃO	**15**

1. INTRODUÇÃO

Este relatório apresenta a realização das quatro atividades práticas propostas na disciplina de **Frameworks para Desenvolvimento de Software**. O trabalho integrou Java, Spring MVC, servidor Apache Tomcat, HTML5, CSS, Bootstrap, JavaScript, jQuery e o serviço ViaCEP, registrando por imagens as etapas de configuração, implementação e execução.

O roteiro e o manual de instalação foram utilizados como referência técnica. Como o manual descreve um ambiente antigo baseado em Windows, NetBeans, Java 8 e Tomcat 8, a prática foi adaptada ao macOS com ferramentas atuais e compatíveis: Visual Studio Code como ambiente de desenvolvimento, OpenJDK 26, Apache Maven e Spring Boot com Tomcat incorporado. A adaptação preservou os objetivos pedagógicos, sem depender de componentes indisponíveis ou obsoletos.

As evidências apresentadas foram produzidas pela execução local do projeto. Os formulários foram testados no navegador, a consulta de CEP retornou dados reais do serviço ViaCEP e as validações foram verificadas tanto no servidor Spring MVC quanto no cliente com jQuery.

Objetivo geral

Aplicar frameworks e bibliotecas no desenvolvimento de aplicações web, compreendendo a integração entre servidor, interface, validação de dados e consumo de serviços externos.

2. DESENVOLVIMENTO

2.1 Ambiente e adaptação para macOS

Antes da implementação, o ambiente foi conferido e complementado. O Java e o Visual Studio Code já estavam instalados. O Maven foi instalado pelo Homebrew para gerenciar dependências, testes e execução. O Tomcat externo do roteiro foi substituído pelo Tomcat incorporado ao Spring Boot, uma solução equivalente e mais atual para projetos Spring no macOS.

Componente do roteiro	Solução utilizada no macOS
Java/JDK 8	OpenJDK Temurin 26.0.2.1, compilação compatível com Java 21
NetBeans	Visual Studio Code já instalado
Tomcat 8 externo	Apache Tomcat 10.1.46 incorporado ao Spring Boot
Gerenciamento manual	Apache Maven 3.9.16
Spring Web MVC	Spring Boot 3.5.6, Spring MVC e Thymeleaf

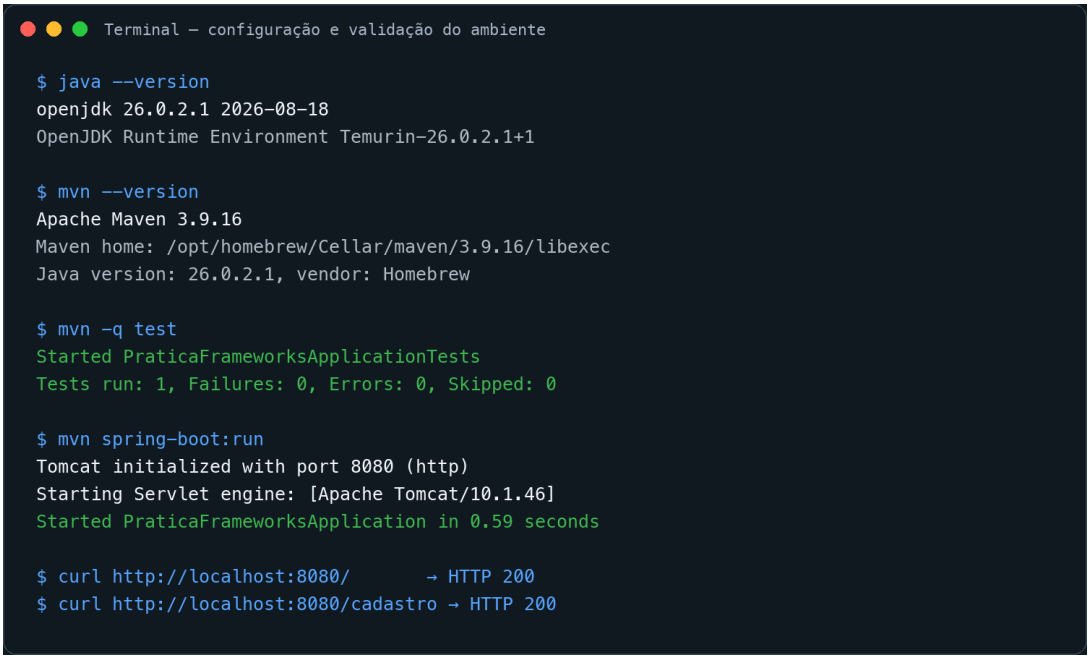


Figura 1 - Verificação das versões, testes automatizados e inicialização do servidor local.

2.2 Atividade 1 - Spring MVC e Tomcat

A primeira atividade solicitou a configuração do servidor Tomcat e a criação de uma aplicação Spring Web MVC. Foi criado um projeto Maven com dependências de Spring Web, Thymeleaf e validação. A classe principal inicia o contêiner Spring e o servidor Tomcat incorporado.

Classe principal

```
@SpringBootApplication
public class PraticaFrameworksApplication {
    public static void main(String[] args) {
        SpringApplication.run(PraticaFrameworksApplication.class, args);
    }
}
```

Controlador MVC

```
@Controller
public class PageController {
    @GetMapping("/")
    public String inicio() {
        return "index";
    }

    @GetMapping("/cadastro")
    public String cadastro(Model model) {
        model.addAttribute("cadastro", new CadastroForm());
        return "cadastro";
    }
}
```

A separação entre classe de inicialização, controlador e templates segue o padrão MVC: o controlador recebe a requisição e seleciona a visão; o Spring gerencia o ciclo da aplicação; e o Thymeleaf produz a página HTML.

2.2 Atividade 1 - Execução

O projeto foi iniciado com o comando **mvn spring-boot:run**. O log confirmou o Tomcat na porta 8080 e a página inicial respondeu com código HTTP 200. A tela também identifica as tecnologias utilizadas e oferece acesso às demais atividades.

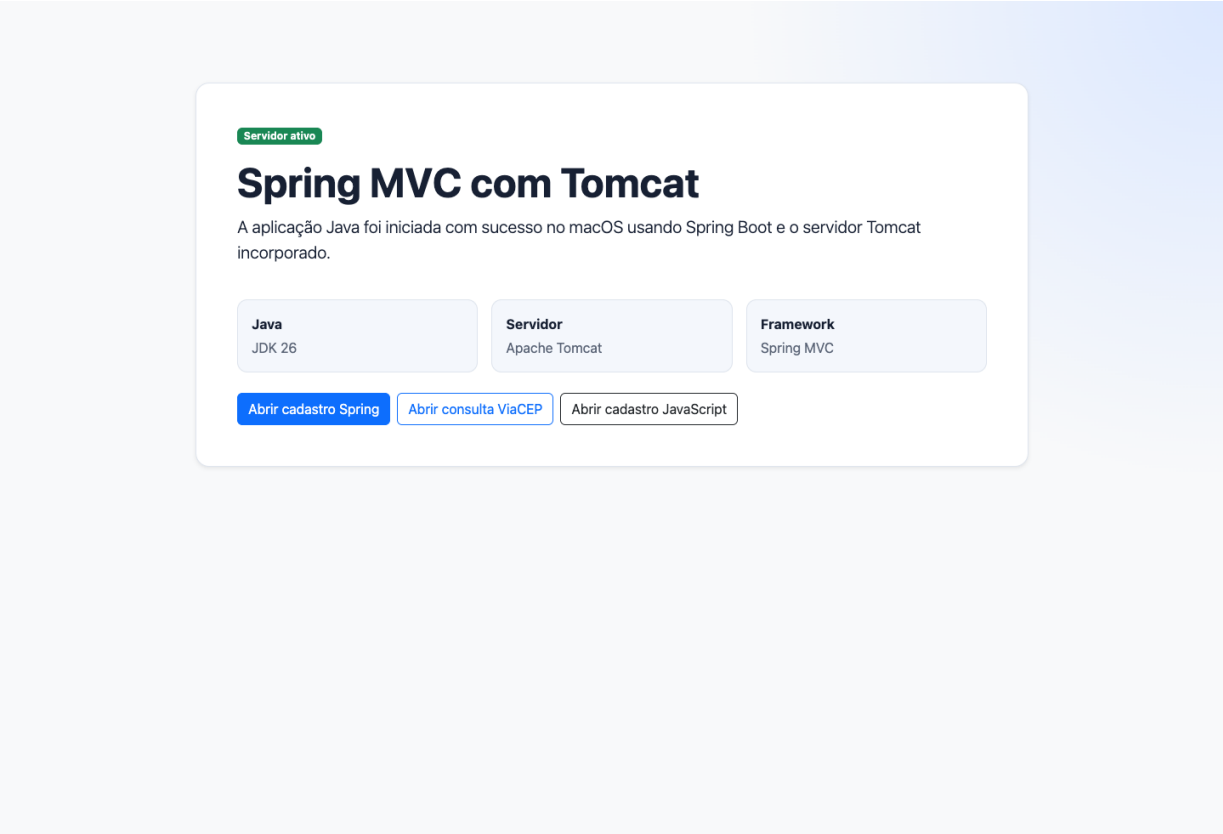


Figura 2 - Aplicação Spring MVC executada no Tomcat local em http://localhost:8080/.

Além da verificação visual, foi executado um teste de carregamento do contexto Spring. O teste foi concluído sem falhas, comprovando que as dependências e os componentes principais foram configurados corretamente.

```
$ mvn -q test
Started PraticaFrameworksApplicationTests
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

2.3 Atividade 2 - Consulta de CEP

A segunda atividade solicitou um formulário HTML5 com campos de endereço e integração com o ViaCEP. A interface contém CEP, rua, bairro, cidade, estado, número e complemento. O JavaScript remove caracteres não numéricos, valida os oito dígitos e realiza uma requisição assíncrona ao serviço.

Trecho da consulta assíncrona

```
async function buscarEndereco() {
    const cep = cepInput.value.replace(/\D/g, '');
    if (cep.length !== 8) {
        setMessage('Informe um CEP com 8 dígitos.', 'warning');
        return false;
    }

    const response = await fetch(
        `https://viacep.com.br/ws/${cep}/json/`
    );
    const data = await response.json();
    fields.rua.value = data.logradouro || '';
    fields.bairro.value = data.bairro || '';
    fields.cidade.value = data.localidade || '';
    fields.estado.value = data.uf || '';
}
```

A consulta é disparada pelo botão “Buscar”. Em caso de sucesso, os campos do endereço são preenchidos automaticamente. Erros de formato, resposta HTTP ou CEP inexistente são apresentados ao usuário por uma mensagem contextual.

2.3 Atividade 2 - Resultado

Foi informado o CEP **86010-000**. A resposta do ViaCEP preencheu o logradouro, o bairro, a cidade e a unidade federativa. Os campos número e complemento permaneceram editáveis porque não fazem parte da identificação pública do CEP.

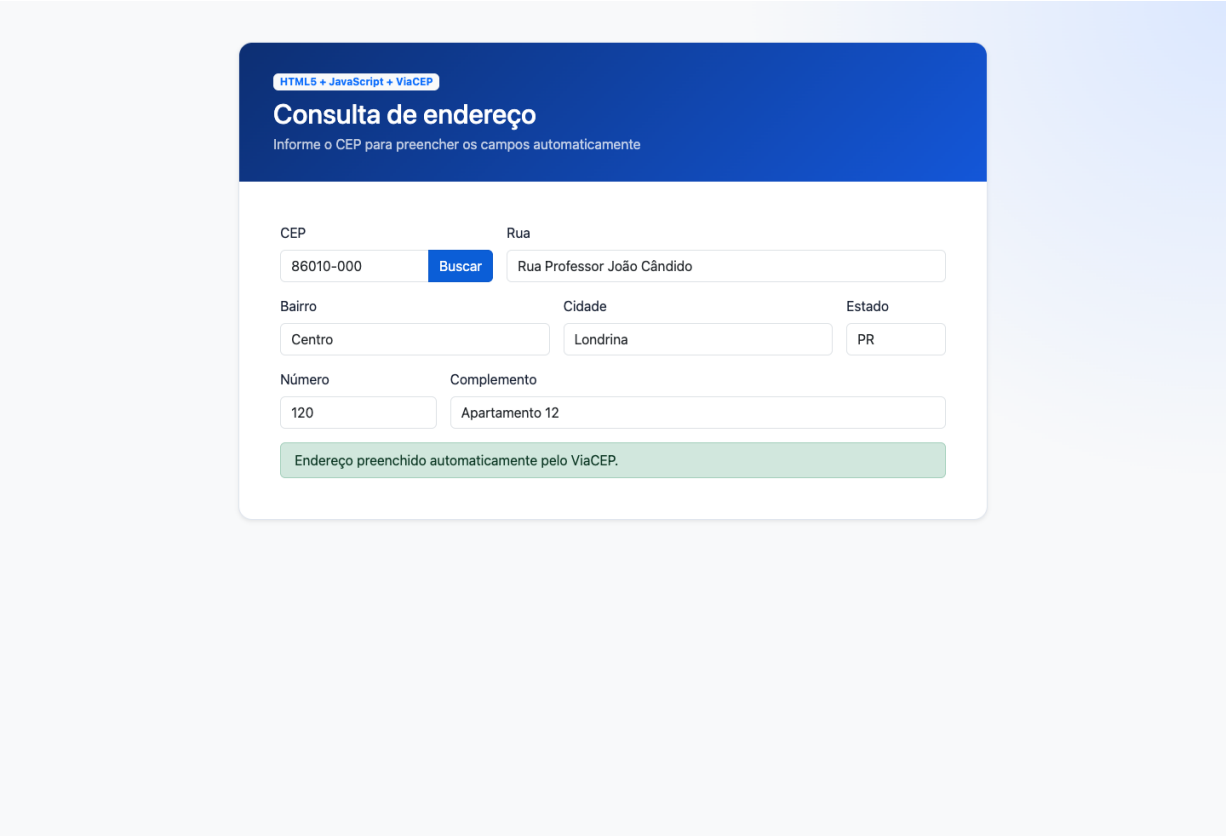


Figura 3 - Consulta do CEP 86010-000 e preenchimento automático do endereço pelo ViaCEP.

Campo	Valor retornado/registrado
CEP	86010-000
Rua	Rua Professor João Cândido
Bairro	Centro
Cidade/UF	Londrina/PR
Número e complemento	Informados pelo usuário

2.4 Atividade 3 - Formulário Spring MVC

Na terceira atividade, o formulário completo de cadastro foi integrado ao Spring MVC. Foi criada uma classe de formulário com regras de validação e um método POST no controlador. Os campos obrigatórios incluem nome, sobrenome, e-mail, senha, CEP, rua, bairro, cidade, estado e número; o complemento é opcional.

Validação no servidor

```
@PostMapping("/cadastro")
public String processarCadastro(
        @Valid @ModelAttribute("cadastro") CadastroForm cadastro,
        BindingResult bindingResult,
        RedirectAttributes redirectAttributes) {
    if (bindingResult.hasErrors()) {
        return "cadastro";
    }
    redirectAttributes.addFlashAttribute(
        "sucesso", "Cadastro validado com sucesso pelo Spring MVC."
    );
    return "redirect:/cadastro";
}
```

```
@NotBlank private String nome;
@NotBlank private String sobrenome;
@Email @NotBlank private String email;
@Size(min = 8) private String senha;
@Pattern(regexp = "\\d{5}-?\\d{3}") private String cep;
```

Essa validação no servidor impede que dados incompletos ou inconsistentes sejam processados mesmo quando a validação do navegador é contornada.

2.4 Atividade 3 - Resultado

Inicialmente, o formulário foi aberto vazio para verificar a organização e a responsividade dos campos. Em seguida, todos os dados foram preenchidos e enviados ao controlador Spring. O retorno confirmou que as regras do servidor foram atendidas.

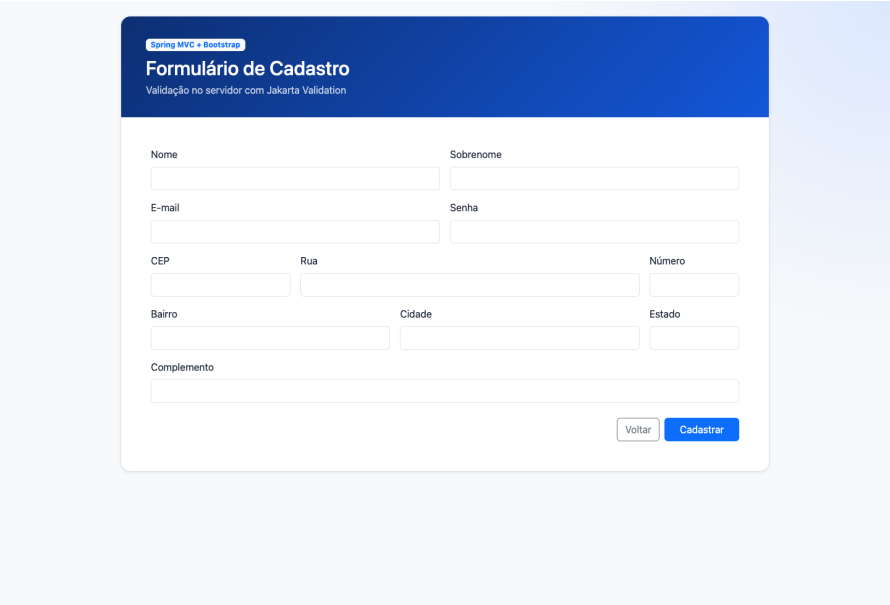


Figura 4 - Formulário de cadastro renderizado pelo Spring MVC e Thymeleaf.

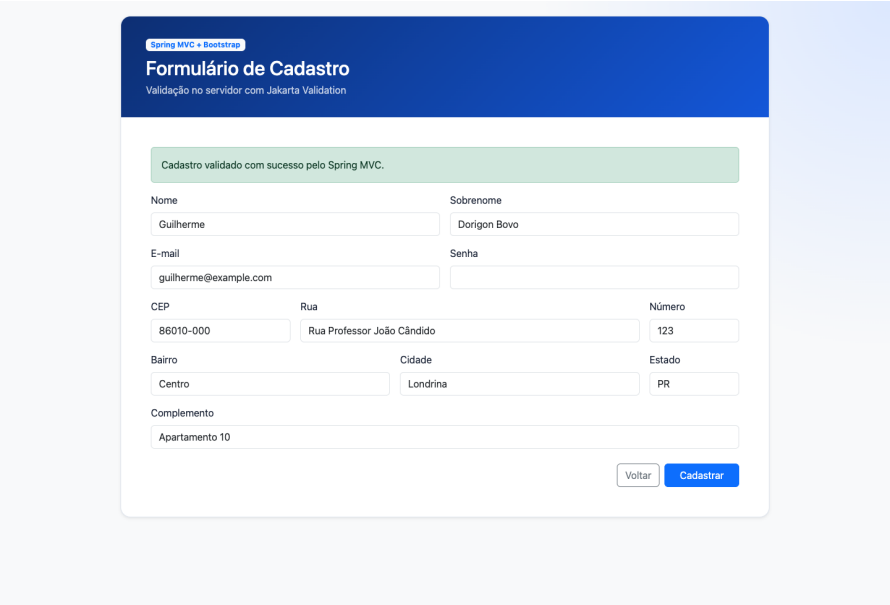


Figura 5 - Cadastro preenchido e validado com sucesso pelo servidor Spring MVC.

2.5 Atividade 4 - jQuery e ViaCEP

A quarta atividade reuniu Bootstrap, jQuery e ViaCEP em uma página de cadastro executada no navegador. O Bootstrap foi utilizado no layout responsivo, o jQuery no tratamento do envio e das classes de erro, e o JavaScript assíncrono na consulta do endereço.

Validação com jQuery

```
$(function () {
    const $form = $('#clienteForm');
    $('#cep').trigger('blur');

    $form.on('submit', function (event) {
        event.preventDefault();
        const emailValido = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
            .test($('#email').val().trim());
        const camposPreenchidos = $form.find('[required]')
            .toArray()
            .every(field => $(field).val().trim().length > 0);

        if (emailValido && camposPreenchidos) {
            $('#mensagem').attr('class', 'alert alert-success')
                .text('Cadastro validado com sucesso.');
        }
    });
});
```

A biblioteca jQuery foi armazenada no próprio projeto para que a validação não dependa da disponibilidade de uma CDN. A integração do CEP reutiliza a mesma função assíncrona da atividade anterior.

2.5 Atividade 4 - Resultado

Ao abrir a página, a consulta do CEP foi acionada e os campos de endereço foram preenchidos. Depois, o formulário foi enviado. O jQuery verificou os campos obrigatórios e o formato do e-mail, exibindo uma confirmação de sucesso sem recarregar a página.

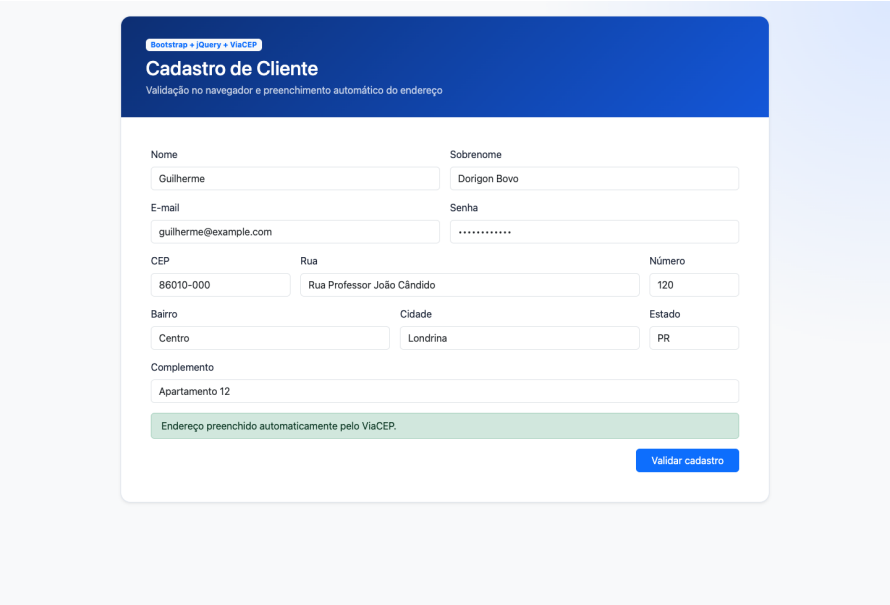


Figura 6 - Formulário Bootstrap com endereço preenchido automaticamente pelo ViaCEP.

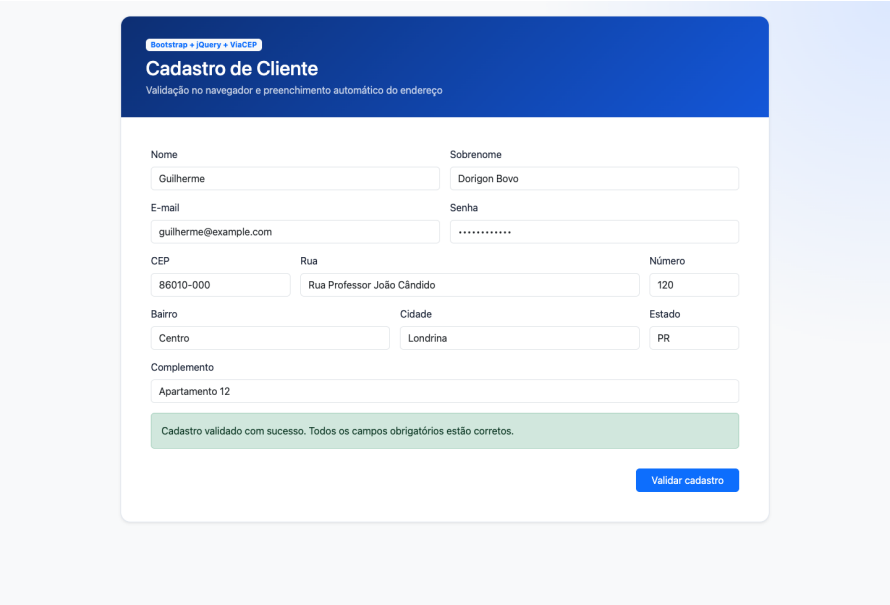


Figura 7 - Mensagem de sucesso após a validação dos campos com jQuery.

2.6 Análise dos resultados

As quatro atividades foram concluídas e executadas em um único projeto, permitindo observar a função de cada camada. O Spring MVC organizou as rotas e a validação no servidor; o Thymeleaf ligou o modelo Java ao HTML; o Tomcat recebeu as requisições; o Bootstrap estruturou a apresentação; o jQuery simplificou a manipulação dos campos; e o ViaCEP forneceu dados externos por meio de uma API.

Atividade	Resultado verificado
1 - Spring MVC/Tomcat	Aplicação iniciada na porta 8080, HTTP 200 e teste de contexto aprovado
2 - HTML/JavaScript/ViaCEP	CEP consultado e endereço preenchido automaticamente
3 - Spring MVC/Bootstrap	Formulário renderizado, enviado e validado no servidor
4 - jQuery/ViaCEP	Validação no cliente e confirmação sem recarregamento da página

Considerações técnicas

A substituição do Tomcat externo pelo servidor incorporado reduziu a configuração manual sem alterar o funcionamento HTTP da aplicação. A validação foi aplicada em duas camadas: no navegador, para resposta imediata ao usuário, e no servidor, para preservar a integridade dos dados. A consulta do ViaCEP foi tratada como operação assíncrona e incluiu mensagens de carregamento, sucesso e erro.

Como melhoria futura, os dados cadastrados poderiam ser persistidos em um banco de dados e protegidos por autenticação. Também seria possível adicionar testes automatizados dos controladores e simulações do serviço ViaCEP.

3. CONCLUSÃO

A prática demonstrou, de forma integrada, como frameworks e bibliotecas contribuem para o desenvolvimento de aplicações web. O projeto foi configurado e executado com sucesso no macOS, usando ferramentas atuais que atendem aos mesmos objetivos do ambiente indicado no roteiro.

A aplicação Spring MVC funcionou no Apache Tomcat incorporado, os formulários foram apresentados com Bootstrap, a validação foi realizada no servidor e no navegador, e a API ViaCEP preencheu os campos de endereço corretamente. Os testes automatizados e as respostas HTTP confirmaram a estabilidade mínima da configuração.

A atividade também evidenciou a importância de separar responsabilidades. Enquanto o servidor recebe requisições e valida regras essenciais, o navegador cuida da experiência de preenchimento e do retorno visual. Essa combinação resulta em uma solução mais organizada, reutilizável e segura.

Referências utilizadas

ANHANGUERA. *Roteiro de Aula Prática: Frameworks para Desenvolvimento de Software*. Material didático fornecido na disciplina.

ANHANGUERA. *Manual de Instalação do Ambiente de Desenvolvimento*. Material didático fornecido na disciplina.

Documentação técnica consultada durante a implementação: Spring Boot, Spring MVC, Thymeleaf, Bootstrap, jQuery e ViaCEP.

Projeto executado localmente e evidências capturadas em 17 de setembro de 2026.